

Compiler Construction

Introduction

Ghiles Ziat & Paul Jeanmaire

This Course

60 hours:

- 20 hours CM
- 40 hours TD/TP

Teachers:

- ghiles.ziat@epita.fr
- paul.jeanmaire@lip6.fr

Evaluation:

- No exam
- Project (2 people)
- Oral questions at last TP session

Languages:

- Java
- OCaml

Context: Natural follow-up of DLP & Natural preparation for TAS (M2)

What's this course about?

Study the Middle-End / Backend of a Compiler

Bridges languages and hardware:

- From high-level code to machine instructions
- Makes computer architecture concrete

Theoretical interest:

- Control-flow graphs, data-flow analysis
- Register allocation, instruction scheduling

Skills:

- Used in compilers, OS kernels, embedded systems
- Deep understanding of how programs actually run

What's this course about concretely?

```
let x = 50 in
(
  while x < 52 do
  (
    x = x + 1
  );
  x
)
```

→

```
li s1, 50
mv s2, s1
L1:
li s1, 52
blt s2, s1, L4
l0:
j L5
L4:
li s1, 1
mv s1, s1
j L6
L5:
mv s1, x0
L6:
bne s1, x0, L2
l1:
j L3
L2:
addi s1, s2, 1
mv s2, s1
j L1
L3:
ret
```

ILP → RiscV

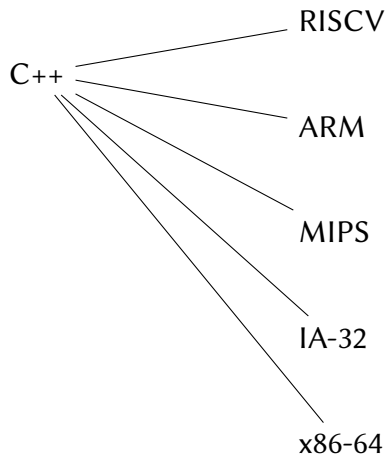
ILP:

- High level
- Dynamic
- ML flavour

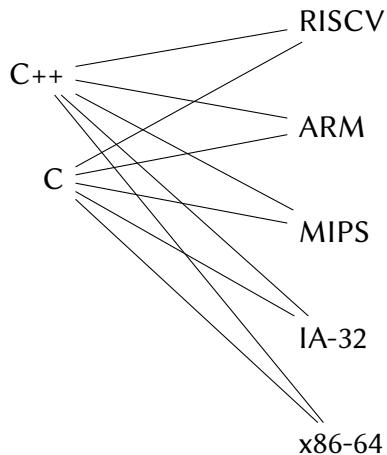
Riscv:

- RISC ISA
- “Reduced” / simple
- Open-sourced

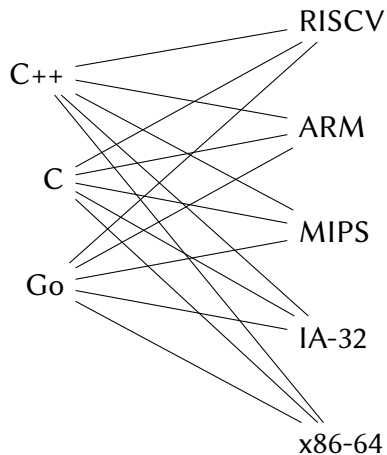
Retargetable Compilers (1/2)



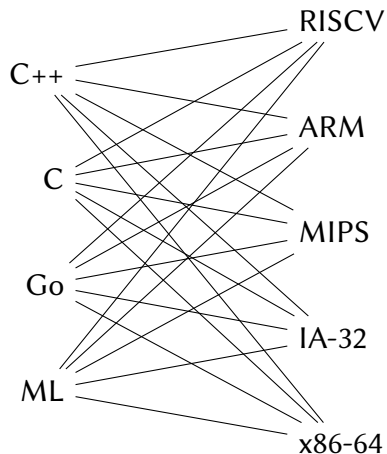
Retargetable Compilers (1/2)



Retargetable Compilers (1/2)



Retargetable Compilers (1/2)



Goal of middle end

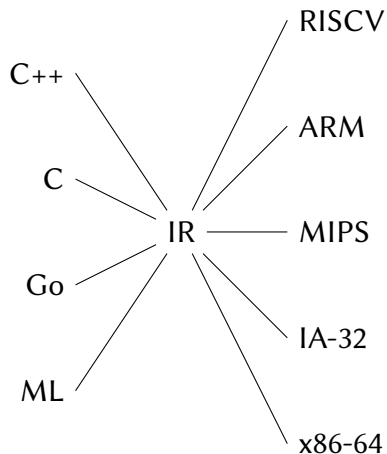
An intermediate representation (IR) is not *required*.

But

preferable to:

- simplify software architecture
- reduce code complexity
- factorizing optimisations

Retargetable Compilers (2/2)

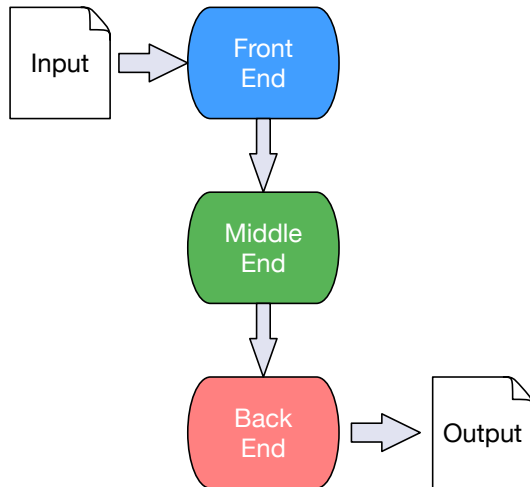


Tree Intermediate Representation

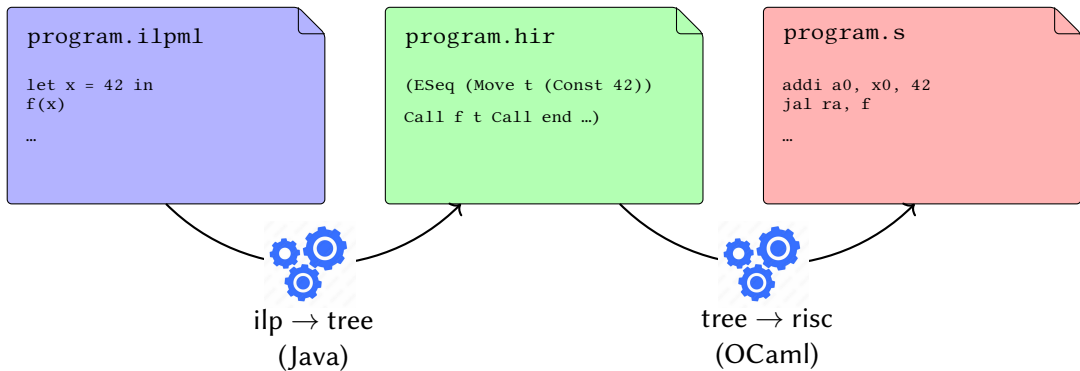
- Low-level IRs designed as “portable assembly,” (CIL, LLVM IR ...).
- Java bytecode is a shared intermediate language for all JVM-targeting compilers.
- GCC internally relies on multiple intermediate representations:
 - ▶ RTL (Register Transfer Language)
 - ▶ GENERIC (tree-based)
 - ▶ GIMPLE (SSA-based)

Tree IR: Tree representations model programs as abstract syntax trees where each node represents an operation and its operand

General Compiler Structure

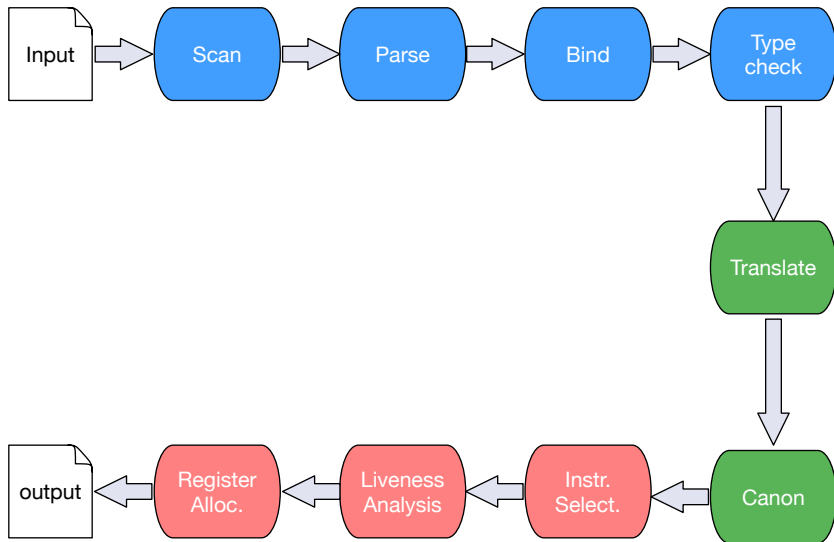


Project



Java + OCaml = The Jamel Project!

General Compiler Structure Zoomed In





Problems:

- ILP is dynamically typed: variables/functions may have multiple possible types.
- Tree must know type of variables statically
- Supports multiple types: `int`, `float`, `bool`, `string`, functions, objects.
- Tree IR only has `int` and `float`.

Solution:

- Implement a **static type inference system**.
- Assign types to every expression in the ILP program.
- Detect type errors early (e.g., `'2 * "hello"'`).

Example:

```
let x = 27;           // inferred as int
let y = deuxfois(x); // function typed as int -> int
```




Problems:

- ILP allows overloading: e.g., `print(int)` vs `print(string)`.
- Tree must know the exact function/operator to call.

Solution:

- Remove ambiguities caused by multiple functions/operators with the same name.
- Resolve overloads based on argument types inferred in the previous stage.
- Monomorphize: build unique versions of each function for its argument types.

Example:

```
print(deuxfois(27))  
// After resolution:  
print_int(deuxfois_int(27))
```



Problems:

- ILP has string literals, Tree has memory accesses
- ILP has control structure, Tree has jumps

Solutions:

- String are collected and emitted as headers
- Control flow is made explicit using labels and jumps.

Example:

```
function deuxfois(x) { 2 * x }  
// After conversion to Tree IR  
label L2  
move temp rv  
  binop mul const 2 temp i0
```



Problems:

- Tree IR is a tree of expressions; assembly prefers linear list of expression.
- Tree call are unconstrained; assembly has conventions.

Solution:

- Flatten nested expressions into sequences ('ESeq') of moves and computations.
- Maintain correct order and dependencies.

Example:

```
(ESeq (Move temp t (Const 42)) (Call f t))  
// Linearized  
move temp t const 42  
call f temp t
```



Problems:

- Linearized IR contains 2-way conditionals, assemblers and CPUs prefer fall-through execution.
- Control flow is correct but inefficient

Solution:

- Group instructions into straight-line code with a single entry and exit.
- Reorder blocks to maximize fall-through paths.

Example:

```
if c goto L1
goto L2
// After
if !c goto L2
L1:
```



Map linearized IR to target-specific instructions (e.g., RISC-V).

- Each IR operation is replaced by one or more machine instructions.
- Consider instruction constraints: registers, immediates, memory.

Example:

```
// IR  
move temp t10 binop mul const 2 temp i5
```

```
// RISC-V  
addi t44, x0, 2  
mul t10, t44, i5
```



Problems:

- Limited number of registers.
- Must respect calling conventions.

Solution:

- Use graph coloring or linear scan algorithms.
- Spill registers to stack if needed.

Example:

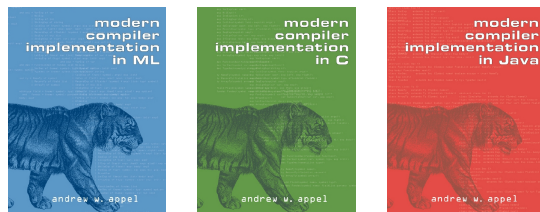
```
// Infinite temporaries
```

```
addi t44, x0, 2  
mul t10, t44, i5
```

```
// Registers t1-7
```

```
addi t2, x0, 2  
mul t4, t2, t1
```

Ressources



RISCV cheatsheet:

https://llp.mit.edu/6190/_static/S24A/resources/references/riscv_isa_reference.pdf